

wms_rcs_interface_requirements

WES 对 WMS/RCS 接口需求分析

版本: 3.0

日期: 2026-03-14

依据: SRS.md, third_party_integration_whitepaper.md, workline_plugin_architecture_design.md

范围: 基础数据需求 + 装箱区 WMS/RCS 接口

1. 设计原则

1.1 系统定位

系统	职责	接口方向
WMS	库存主数据、账务管理、RCS 调度	上位系统
WES	执行协调、策略引擎、设备编排	中间层
RCS	AGV/CTU 调度、路径规划	由 WMS 统一调度

1.2 集成边界（SRS.md § 3 本阶段集成边界）

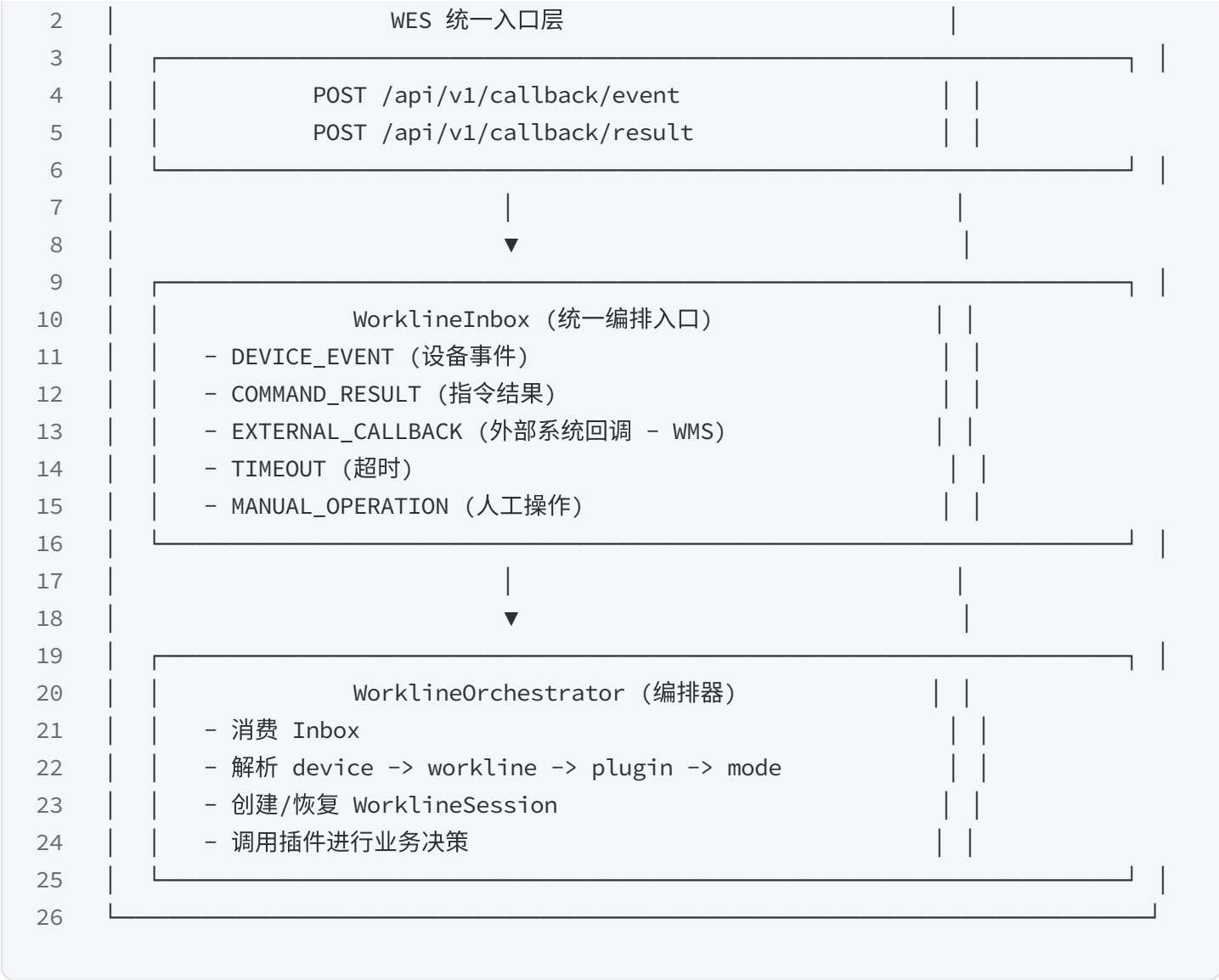
1. **RCS 调度:** 仍由 WMS 统一调度。WES 生成搬运需求并提交给 WMS，由 WMS 调用 RCS 并将结果/事件回传 WES。
2. **PDA 交互:** PDA 仅对接 WMS 应用。WES 如需感知 PDA 结果/事件，由 WMS 推送/同步。
3. **自动化设备:** 所有自动化设备只通过 WES 接入，WMS 不直连设备。
4. **标签打印:** WES 生成打印模板/ZPL。若为自动打印设备则由 WES 下发；若为人工/非自动打印，则 WMS 获取模板后完成打印并回执结果。

1.3 核心架构原则（遵循 workline_plugin_architecture_design.md）

WMS 作为"外部系统"调用 WES 时，必须遵循白皮书定义的标准回调接口：

代码块

1



1.4 数据原则

- **WES 不同步基础数据:** 所有需要用到的基础数据向 WMS 请求
- **WMS 是库存唯一真实源:** 所有库存变动必须在 WMS 端事务提交成功后，物理动作方可视为完成
- **不过度设计:** WES 只关注需要它完成的功能
- **统一入口:** 所有外部输入通过 `callback/event` 和 `callback/result` 进入 WES

2. 基础数据需求

说明: WES 不同步基础数据，按需动态查询 WMS。所有接口遵循 RESTful 规范，采用异步回调机制。

2.1 物料主数据 (Material Master Data)

业务背景: WES 在装箱校验、分箱算法、发料决策等场景需要实时获取物料属性信息。

2.1.1 查询单个物料信息

项目	说明
接口地址	GET /api/wms/materials/{material_id}
请求方法	GET
用途	PKG 校验时验证物料存在性，获取物料尺寸/厚度等属性
调用场景	- SMT 智能装箱：视觉扫描 PKG 后校验物料合法性

- 分箱算法：计算料盘可堆叠数量
- 发料决策：判断物料是否为 MSD/高值物料 |
| 入参 | - material_id (Path): 物料编码，如 CAP001 |
| 响应码 | - 200 : 成功
- 404 : 物料不存在 |

请求示例:

代码块

```
1 GET /api/wms/materials/CAP001|Vendor001 HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1 {
2   "code": 200,
3   "data": {
4     "material_id": "CAP001",           // 物料编码 (主键)
5     "material_name": "电容 0402",     // 物料名称
6     "vendor": "V0001",                // 默认供应商编码
7     "standard_dims": "7inch",         // 标准尺寸规格: 7inch/13inch/15inch
8     "standard_thickness": 15.0,       // 标准厚度 (mm), 用于分箱堆叠计算
9     "is_msd": false,                // 是否湿敏物料, true 需要干燥柜存储
10    "is_auto": true,                // 是否走自动流程
11    "is_high_value": false,         // 是否高值物料, true 需要双人复核
12    "is_precious": false,          // 是否贵重物料, true 送贵重品仓
13    "is_pcb": false,                // 是否 PCB 物料, true 送 PCB 专区
14    "is_irregular": false,         // 是否异形物料, true 走人工线
15    "material_type": "ELECTRONIC", // 物料类型:
    ELECTRONIC/PCB/MECHANICAL/MSD
```

```
16      "lc_cycle": 30, // 保质期天数，用于 FEFO 发料策略
17      "floor_life": 168 // 湿敏物料暴露时限 (小时)，仅 MSD 物料有效
18  }
19 }
```

设计说明:

- material_type 用于路由决策: ELECTRONIC → 装箱区, MSD → 干燥柜, PCB → PCB 专区, MECHANICAL → 平库
- standard_thickness 允许 ±10% 偏差, WES 在视觉识别后校验实际厚度
- floor_life 用于计算 MSD 物料剩余暴露时间: remaining = floor_life - (now - open_time)

2.1.2 批量查询物料信息

项目	说明
接口地址	GET /api/wms/materials?ids={id1,id2,...}
请求方法	GET
用途	分箱算法需要多个物料的尺寸/厚度，一次性获取减少网络往返
调用场景	- SMT 智能装箱：当前 GRN 下所有物料的分箱预计算

- 发料波次：多个工单所需物料的批量属性查询 |
| 入参 | - ids (Query): 物料编码列表，逗号分隔，如 CAP001,RES002,IC003 |
| 响应码 | - 200 : 成功
- 400 : 参数格式错误 |

请求示例:

```
代码块
1 GET /api/wms/materials?ids=CAP001|Vendor001,RES002|Vendor002,IC003|Vendor003
  HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
2  "code": 200,
3  "data": [
4    {
5      "material_id": "CAP001",
6      "material_name": "电容 0402",
7      "standard_dims": "7inch",
8      "standard_thickness": 15.0,
9      "material_type": "ELECTRONIC"
10     "vendor": "V0001", // 默认供应商编码
11   },
12   {
13     "material_id": "RES002",
14     "material_name": "电阻 0603",
15     "standard_dims": "7inch",
16     "standard_thickness": 12.5,
17     "vendor": "V0001", // 默认供应商编码
18   },
19   {
20     "material_id": "IC003",
21     "material_name": "IC 芯片 SOP-8",
22     "standard_dims": "13inch",
23     "standard_thickness": 25.0,
24     "vendor": "V0001", // 默认供应商编码
25   }
26 ],
27 "not_found": [] // 可选: 返回未找到的物料编码列表
28 }
```

2.2 仓库/区域/地码 (Warehouse / Zone / Location)

业务背景: WES 需要查询区域和地码信息用于设备归属解析、货架位置管理、搬运任务起点/终点定义。

2.2.1 查询区域列表

项目	说明
接口地址	GET /api/wms/zones
请求方法	GET
用途	查询所有区域信息，用于设备归属解析和任务路由

- 任务调度：根据区域属性决定 RCS 搬运路径
- 状态监控：按区域统计设备在线率 |
| 入参 | 可选查询参数：
- `type` (Query): 区域类型，如 `KITTING` / `STORAGE` / `IQC`
- `status` (Query): 区域状态，如 `ACTIVE` / `DISABLED` |
| 响应码 | - `200` : 成功
- `500` : 服务器错误 |

请求示例:

代码块

```
1 GET /api/wms/zones?type=KITTING HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1 {
2   "code": 200,
3   "data": [
4     {
5       "zone_code": "KITTING_AREA",           // 区域编码（主键）
6       "zone_name": "装箱区",                 // 区域名称
7       "zone_type": "KITTING",                // 区域类型：
8       KITTING/STORAGE/IQC/DOCK/RETURN
9       "status": "ACTIVE",                    // 状态：ACTIVE/DISABLED
10      "parent_zone": "SMT_ZONE",              // 父区域编码（可选，用于层级结构）
11      "allowed_rack_types": ["SINGLE_LAYER"],  // 允许存放的货架类型
12      "max_concurrent_tasks": 10,             // 区域最大并发任务数
13      "description": "SMT 物料装箱作业区"    // 区域描述
14    },
15    {
16      "zone_code": "SMT_STORAGE",
17      "zone_name": "SMT 自动化立库区",
18      "zone_type": "STORAGE",
19      "status": "ACTIVE",
20      "allowed_rack_types": ["FIVE_LAYER"],
21      "max_concurrent_tasks": 50
22    }
23  ]
24 }
```

```
22     ]
23 }
```

2.2.2 查询区域内地码

项目	说明
接口地址	GET /api/wms/locations?zone={zone_code}
请求方法	GET
用途	查询指定区域内的所有地码，用于货架位置管理和搬运任务
调用场景	- 货架位置初始化：建立 Rack -> Location 映射

- RCS 搬运：指定搬运任务的起点/终点
- 设备注册：绑定设备所属地码 |
| 入参 | - zone (Query, 必填): 区域编码，如 KITTING_AREA
- location_type (Query, 可选): 地码类型，如 BUFFER / WORK_STATION / STORAGE |
| 响应码 | - 200 : 成功
- 400 : 参数错误
- 404 : 区域不存在 |

请求示例:

代码块

```
1 GET /api/wms/locations?zone=KITTING_AREA&location_type=BUFFER HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1 {
2   "code": 200,
3   "data": [
4     {
5       "location_code": "KITTING_AREA_LOC_01", // 地码编码 (主键)
6       "location_name": "装箱区 -01 号位",    // 地码名称
7       "zone_code": "KITTING_AREA",          // 所属区域

```

```

8      "location_type": "BUFFER",                // 类型:
      BUFFER/WORK_STATION/STORAGE/DOCK
9      "coordinates": {                        // 物理坐标 (用于 RCS 导航)
10         "x": 100.5,
11         "y": 200.3,
12         "z": 0
13     },
14     "rack_capacity": 4,                      // 区域可放置货架数
15     "current_rack_count": 0,                 // 当前货架数
16     "status": "AVAILABLE",                  // 状态:
      AVAILABLE/OCCUPIED/DISABLED
17     "allowed_rack_types": ["SINGLE_LAYER", "FIVE_LAYER"] // 区域允许的货架类
      型, 单层、五层、转运、退货....
18 },
19 {
20     "location_code": "KITTING_AREA_INPUT",
21     "location_name": "装箱区 - 上线口",
22     "zone_code": "KITTING_AREA",
23     "location_type": "WORK_STATION",
24     "status": "AVAILABLE",
25     "rack_capacity": 1
26 }
27 ]
28 }

```

设计说明:

- `coordinates` 用于 RCS 导航，WES 仅作为透传，不解析具体坐标值
- `rack_capacity` 用于 WES 判断区域是否可容纳新货架
- `status=DISABLED` 的地码不得作为搬运任务目的地

2.3 货架/料箱主数据 (Rack / Bin Master Data)

业务背景: WES 需要查询货架和料箱属性用于分箱决策、混合入库策略、发料波次计算。

2.3.1 查询单个货架信息

项目	说明
接口地址	GET /api/wms/racks/{rack_id}
请求方法	GET

用途	查询货架详细属性，用于货架状态初始化和任务决策
调用场景	- 货架到达通知：验证货架类型和属性

- 混合入库：判断货架容量和当前负载
- 发料决策：查询货架位置和优化建议 |
| 入参 | - rack_id (Path): 货架编码，如 RACK-001 |
| 响应码 | - 200 : 成功
- 404 : 货架不存在 |

请求示例:

代码块

```
1 GET /api/wms/racks/RACK-001 HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1 {
2   "code": 200,
3   "data": {
4     "rack_id": "RACK-001",           // 货架编码 (主键)
5     "rack_type": "SINGLE_LAYER",     // 货架类型:
6     SINGLE_LAYER/FIVE_LAYER/PRODUCTION/RETURN
7     "rack_name": "单层货架 -001",   // 货架名称
8     "status": "AVAILABLE",          // 状态: AVAILABLE/LOCKED/FULL/DISABLED
9     "current_location": "KITTING_AREA_LOC_01", // 当前所在地码
10    "capacity": {
11      "total_slots": 4,               // 总储位数 (单层=4, 五层=20), 即货架能放多
12      少个箱子
13      "occupied_slots": 0,            // 已占用储位数, 已经放了多少个箱子
14      "total_depth_mm": 400,          // 总深度 (mm)
15      "used_depth_mm": 0              // 已用深度 (mm)
16    },
17    "attributes": {
18      "has_a_side": false,            // 是否有 A 面 (单层=false, 五层/生产/退货
19      =true)
20      "has_b_side": false,            // 是否有 B 面
21      "max_bin_weight_kg": 50         // 最大载重 (kg)
22    },
23    "side_a": {                       // A 面储位 (仅当 has_a_side=true)
```

```

21      // 每一层放了哪些箱子
22      "layer_1": [{"bin_id": null, "status": "EMPTY"}, {"bin_id": null,
    "status": "EMPTY"}],
23      "layer_2": {"bin_id": "BIN-001", "status": "OCCUPIED"},
24      "layer_3": {"bin_id": null, "status": "EMPTY"},
25      "layer_4": {"bin_id": null, "status": "EMPTY"},
26      "layer_5": {"bin_id": null, "status": "EMPTY"}
27  },
28  "side_b": {                                // B 面储位 (仅当 has_b_side=true)
29      "layer_1": {"bin_id": null, "status": "EMPTY"},
30      "layer_2": {"bin_id": null, "status": "EMPTY"},
31      "layer_3": {"bin_id": null, "status": "EMPTY"},
32      "layer_4": {"bin_id": null, "status": "EMPTY"},
33      "layer_5": {"bin_id": null, "status": "EMPTY"}
34  },
35  "metadata": {
36      "created_at": "2026-01-01T00:00:00Z",
37      "last_updated": "2026-03-14T10:30:00Z",
38      "purchase_date": "2025-06-15",
39      "vendor": "RACK-VENDOR-A"
40  }
41  }
42  }

```

设计说明:

- `rack_type` 决定业务处理逻辑: `SINGLE_LAYER` → 装箱区中转, `FIVE_LAYER` → 立库存
- [↑]`status=LOCKED` 表示货架正在搬运中, WES 不得下发新的操作指令
- `side_a/layer_N` 结构仅当 `has_a_side=true` 时返回, 简化响应数据

2.3.2 查询单个料箱信息

项目	说明
接口地址	<code>GET /api/wms/bins/{bin_id}</code>
请求方法	GET
用途	查询料箱详细属性和当前库存, 用于分箱决策和发料拣选
调用场景	- 智能分箱: 查询料箱剩余容量和已存物料

- 发料拣选: 验证料箱位置和可拣选数量

- 退料回流：查询退料货架储位状态 |
| 入参 | - bin_id (Path): 料箱编码, 如 BIN-001 |
| 响应码 | - 200 : 成功
- 404 : 料箱不存在 |

请求示例:

代码块

```
1 GET /api/wms/bins/BIN-001 HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1 {
2   "code": 200,
3   "data": {
4     "bin_id": "BIN-001",           // 料箱编码 (主键)
5     "bin_type": "TYPE_A",         // 料箱类型: TYPE_A (6 个 7 寸储位) /
                                     TYPE_B (2 个 7 寸 +1 个大尺寸)
6     "bin_name": "A 型料箱 -001", // 料箱名称
7     "status": "OCCUPIED",         // 状态: EMPTY/OCCUPIED/FULL/EXCEPTION
8     "current_location": {
9       "rack_id": "RACK-001",      // 所属货架
10      "side": "A",                 // A/B 面
11      "layer": 2,                  // 层号 (1-5)
12      "slot": "SLOT-01"           // 储位编号
13    },
14    "capacity": {
15      "total_slots": 6,             // 总储位数
16      "occupied_slots": 2,          // 已占用储位数
17      "slots": [                   // 各储位详情
18        {
19          "slot_id": "SLOT-01",
20          "slot_type": "7inch",     // 储位类型: 7inch/13inch/15inch/LARGE
21          "status": "OCCUPIED",
22          "current_depth_mm": 150,  // 当前已用深度
23          "remaining_depth_mm": 250, // 剩余可用深度
24          "max_depth_mm": 400,      // 最大深度
25          "pkgs": [                 // 当前料盘列表
26            {
27              "pkg_code": "CAP001-5000-PKG12345-LC01-DC02-V0001",
28              "material_id": "CAP001",
```

```
29         "qty": 5000,
30         "thickness_mm": 15.0,
31         "vendor": "V0001",
32         "lc": "LC01",
33         "dc": "DC02"
34     }
35 ]
36 },
37 {
38     "slot_id": "SLOT-02",
39     "slot_type": "7inch",
40     "status": "EMPTY",
41     "remaining_depth_mm": 400
42 }
43 ]
44 },
45 "metadata": {
46     "created_at": "2026-01-01T00:00:00Z",
47     "last_updated": "2026-03-14T10:30:00Z"
48 }
49 }
50 }
```

设计说明:

- `bin_type` 决定分箱算法: `TYPE_A` 仅能放 7 寸料盘, `TYPE_B` 可放 7 寸 + 大尺寸料盘
- `slot_type` 与料盘尺寸匹配: 7 寸料盘不得放入 `LARGE` 储位, 13/15 寸料盘不得放入 `7inch` 储位
- `pkgs` 数组按放入时间排序, 发料时支持 FIFO/FEFO 策略

2.3.3 按类型查询货架

项目	说明
接口地址	<code>GET /api/wms/racks?type={rack_type}&status={status}&zone={zone_code}</code>
请求方法	GET
用途	批量查询指定类型/状态的货架, 用于混合入库空箱查找、冷热区分析
调用场景	- 混合入库: 查找五层架空箱资源

- 空架补给：查询装箱区可用空架数量
- 冷热区优化：分析各区域货架分布 |
| 入参 | - type (Query, 必填): 货架类型, 如 FIVE_LAYER / SINGLE_LAYER / RETURN
- status (Query, 可选): 货架状态, 如 AVAILABLE / EMPTY / FULL
- zone (Query, 可选): 区域编码, 限定查询范围 |
| 响应码 | - 200 : 成功
- 400 : 参数错误 |

请求示例:

代码块

```
1 GET /api/wms/racks?type=FIVE_LAYER&status=EMPTY&zone=SMT_STORAGE HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1 {
2   "code": 200,
3   "data": [
4     {
5       "rack_id": "RACK-F001",           // 货架编码 (主键)
6       "rack_type": "FIVE_LAYER",       // 货架类型
7       "status": "EMPTY",               // 货架状态
8       "current_location": "SMT_STORAGE_A01", // 当前位置
9       "side_a_empty_slots": 5,          // A 面空储位数
10      "side_b_empty_slots": 5,          // B 面空储位数
11      "accessibility_score": 0.9 // 访问便捷度 (0-1), 用于冷热区优化, 暂时不要, 后
续优化
12    },
13    {
14      "rack_id": "RACK-F002",
15      "rack_type": "FIVE_LAYER",
16      "status": "EMPTY",
17      "current_location": "SMT_STORAGE_A02",
18      "side_a_empty_slots": 5,
19      "side_b_empty_slots": 5,
20      "accessibility_score": 0.85
21    }
22  ],
23  "total_count": 2,
```

```
24     "page": 1,  
25     "page_size": 100  
26 }
```

设计说明:

- `accessibility_score` 用于冷热区优化算法：A 面 > B 面，底层 > 高层，近产线 > 远端
- WES 可对查询结果进行短时缓存 (TTL ≤ 30 秒)，减少 API 调用频率
- 返回数据仅包含基本信息，详细信息需调用单货架查询接口

2.4 GRN 数据 (Goods Receipt Note)

业务背景: WES 在 PKG 校验、装箱任务创建时需要验证 GRN 存在性和物料归属关系。

2.4.1 查询单个 GRN 信息

项目	说明
接口地址	<code>GET /api/wms/grn/{grn_id}</code>
请求方法	GET
用途	查询 GRN 详细信息，用于 PKG 校验时验证 GRN 存在性
调用场景	- PKG 校验：验证料盘是否属于当前 GRN

- 装箱任务：获取 GRN 允许混托类型
- 到货追溯：查询 GRN 来源和收货状态 |
| 入参 | - `grn_id` (Path): GRN 编号，如 `GRN.0001` |
| 响应码 | - `200` : 成功
- `404` : GRN 不存在 |

请求示例:

代码块

```
1 GET /api/wms/grn/GRN.0001 HTTP/1.1  
2 Host: wms-server  
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1  {
2    "code": 200,
3    "data": {
4      "grn_id": "GRN.0001",           // GRN 编号 (主键)
5      "po_number": "PO-2025-001",     // 采购订单号
6      "po_item": "001",               // 行项目号
7      "status": "PARTIAL_RECEIVED",   // 状态:
      // CREATED/PARTIAL_RECEIVED/COMPLETED
8      "dock_location": "DOCK-01",     // 收货码头
9      "arrival_date": "2026-03-14",   // 到货日期
10     "vendor": {
11       "vendor_id": "V0001",
12       "vendor_name": "供应商 A"
13     },
14     "allow_mixed_pallet": true, // 是否允许混托
15     "mixed_pallet_material_types": ["ELECTRONIC"], // 允许混托的物料类型
16     "items": [                       // GRN 明细列表
17       {
18         "material_id": "CAP001",
19         "material_name": "电容 0402",
20         "ordered_qty": 50000,         // 订购数量
21         "received_qty": 25000,       // 已收货数量
22         "remaining_qty": 25000,     // 剩余未收货数量
23
24         "unit": "PCS",
25         "lc": "LC01",                // 批次号
26         "dc": "2026-03-14"          // 生产日期
27       }
28     ],
29     "qc_status": {
30       "sampling_required": true,      // 是否需要抽检
31       "sampled_qty": 0,              // 已抽检数量
32       "passed_qty": 0,               // 合格数量
33       "ng_qty": 0                    // 不合格数量
34     },
35     "metadata": {
36       "created_at": "2026-03-14T08:00:00Z",
37       "updated_at": "2026-03-14T10:30:00Z",
38       "synced_from_sap": true        // 是否已同步 SAP
39     }
40   }
41 }
```

设计说明:

- `status` 用于 WES 判断 GRN 是否可继续收货：`COMPLETED/CLOSED` 不得再绑定
- `remaining_qty` 用于 PDA 绑定校验：`sum(current_qty) <= remaining_qty`
- `mixed_pallet_material_types` 用于混托校验：同栈板物料类型必须在允许列表中

2.4.2 查询 GRN 下的料盘列表

项目	说明
接口地址	<code>GET /api/wms/grn/{grn_id}/packages</code>
请求方法	GET
用途	查询 GRN 下所有已绑定的料盘，用于 PKG 校验和追溯
调用场景	- PKG 校验：视觉扫描后验证料盘是否属于当前 GRN

- 装箱追溯：查询 GRN 下所有料盘的装箱状态
- 收货验收：核对实际收货料盘与 GRN 明细 |
| 入参 | - `grn_id` (Path): GRN 编号
- `status` (Query, 可选): 料盘状态，如 `BOUND` / `KITTED` / `STORED` |
| 响应码 | - `200` : 成功
- `404` : GRN 不存在 |

请求示例:

代码块

```
1 GET /api/wms/grn/GRN.0001/packages HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1 {
2   "code": 200,
3   "data": {
4     "grn_id": "GRN.0001",
5     "total_packages": 10,
6     "packages": [
7       {
```

```

8      "pkg_code": "CAP001-5000-PKG12345-LC01-DC02-V0001", // 六合一码
9      "material_id": "CAP001",
10     "material_name": "电容 0402",
11     "qty": 5000,
12     "vendor": "V0001",
13     "lc": "LC01",
14     "dc": "DC02",
15     "status": "KITTED", // 状态: BOUND/KITTED/STORED/ISSUED
16     "current_location": {
17         "rack_id": "RACK-001", // 所在货架
18         "bin_id": "BIN-001", // 所在料箱
19         "slot_id": "SLOT-01" // 所在料格
20     },
21     "bound_at": "2026-03-14T09:00:00Z",
22     "kitted_at": "2026-03-14T10:30:00Z"
23 },
24 {
25     "pkg_code": "CAP001-5000-PKG12346-LC01-DC02-V0001",
26     "material_id": "CAP001",
27     "qty": 5000,
28     "status": "BOUND", // 已绑定但未装箱
29     "bound_at": "2026-03-14T09:05:00Z"
30 }
31 ],
32 "summary": {
33     "total_qty": 50000,
34     "bound_qty": 25000,
35     "kitted_qty": 20000,
36     "stored_qty": 5000
37 }
38 }
39 }

```

设计说明:

- `pkg_code` 是六合一码: `Material-Qty-Serial-LC-DC-Vendor` , WES 需解析并校验各字段
- `status=KITTED` 表示已完成装箱, 装到了单层货架, `status=STORED` 表示已上架五层货架
- `summary` 用于 WES 判断 GRN 收货进度

2.5 库存查询 (Inventory Query)

业务背景: WES 采用纯代理模式, 不维护库存主数据, 所有库存查询实时透传 WMS。

2.5.1 库存查询接口

项目	说明
接口地址	GET /api/wms/inventory/query
请求方法	GET
用途	查询指定物料/位置的库存信息，用于发料决策和库存校验
调用场景	- 发料波次：查询可用库存并计算可分配数量

- 混合入库：查询五层架空箱资源
- 库存追溯：查询物料当前所在储位 |
| 入参 | - material_id (Query, 可选): 物料编码
- location (Query, 可选): 位置编码 (地码/货架/料箱)
- zone (Query, 可选): 区域编码
- rack_type (Query, 可选): 货架类型，如 FIVE_LAYER
- status (Query, 可选): 库存状态，如 AVAILABLE / RESERVED / QC_HOLD |
| 响应码 | - 200 : 成功
- 400 : 参数错误 |

请求示例:

代码块

```
1 GET /api/wms/inventory/query?material_id=CAP001&zone=SMT_STORAGE HTTP/1.1
2 Host: wms-server
3 Authorization: Bearer <WES_TOKEN>
```

返回数据样例:

代码块

```
1 {
2   "code": 200,
3   "data": {
4     "material_id": "CAP001",
5     "material_name": "电容 0402",
6     "total_qty": 50000,
7     "available_qty": 30000,           // 可用数量 (AVAILABLE 状态)
```

```

8      "reserved_qty": 15000,           // 已预占数量 (RESERVED 状态)
9      "qc_hold_qty": 5000,           // 质检冻结数量 (QC_HOLD 状态)
10     "locations": [                  // 库存分布位置
11         {
12             "location_type": "BIN",
13             "location_id": "BIN-001",
14             "rack_id": "RACK-001",
15             "zone": "SMT_STORAGE",
16             "qty": 20000,
17             "status": "AVAILABLE"
18         },
19         {
20             "location_type": "BIN",
21             "location_id": "BIN-002",
22             "rack_id": "RACK-001",
23             "zone": "SMT_STORAGE",
24             "qty": 10000,
25             "status": "AVAILABLE"
26         },
27         {
28             "location_type": "BIN",
29             "location_id": "BIN-003",
30             "rack_id": "RACK-002",
31             "zone": "SMT_STORAGE",
32             "qty": 15000,
33             "status": "RESERVED",
34             "reserved_for": "WO-2025-001"
35         }
36     ],
37     "cold_chain_info": {
38         "oldest_dc": "DC01",         // 最早生产日期 (用于 FEFO)
39         "oldest_qty": 10000
40     }
41 }
42 }

```

设计说明:

- WES 可对查询结果进行短时缓存 (TTL ≤ 30 秒), 但不改变"WMS 为库存主数据源"的定位
- `available_qty` 用于发料决策: `available_qty >= required_qty` 才能分配
- `cold_chain_info` 用于 FEFO (First Expire First Out) 发料策略

2.5.2 空箱资源查询 (扩展)

项目	说明
接口地址	GET /api/wms/inventory/query? rack_type=five_layer&status=empty
请求方法	GET
用途	查询指定类型的空料箱资源，用于混合入库满箱交换策略
调用场景	- 混合入库：查找五层架空箱用于交换

- 优先交换：判断是否有空箱资源 |
- 查找具有空箱的货架
| 入参 | - rack_type (Query): FIVE_LAYER
- status (Query): EMPTY
- zone (Query, 可选): 限定区域 |
- | 响应码 | - 200 : 成功 |

返回数据样例:

代码块

```
1  {
2    "code": 200,
3    "data": {
4      "rack_type": "FIVE_LAYER",    // 货架类型
5      "status_filter": "EMPTY",    // 对料箱的过渡状态
6      "total_empty_bins": 50,      // 总的空料箱数
7      "bins": [                    // 料箱信息
8        {
9          "bin_id": "BIN-F001-A1",  // 料箱 ID
10         "rack_id": "RACK-F001",   // 所在货架
11         "side": "A",              // 货架面
12         "layer": 1,               // 层
13         "slot": "SLOT-01",        // 位
14         "location": "SMT_STORAGE_A01",
15         "accessibility_score": 0.95
16       },
17       {
18         "bin_id": "BIN-F002-A1",
19         "rack_id": "RACK-F002",
20         "side": "A",
21         "layer": 1,
22         "slot": "SLOT-01",
```

```
23         "location": "SMT_STORAGE_A02",
24         "accessibility_score": 0.9
25     }
26 ]
27 }
28 }
```

2.6 接口调用约束

2.6.1 超时与重试

项目	要求
超时时间	10 秒 (WES 调用 WMS)
重试策略	指数退避 (1s, 2s, 4s)，最多 3 次
失败处理	超过重试次数后触发告警，任务进入 <code>MANUAL_HOLD</code> 状态

2.6.2 缓存策略

数据类型	缓存 TTL	说明
物料主数据	30 秒	可短暂缓存，减少重复查询
货架/料箱状态	10 秒	状态变化频繁，缓存时间较短
库存查询	30 秒	短时缓存，不改变主数据源定位
GRN 数据	60 秒	GRN 数据相对稳定

2.6.3 幂等性要求

- 所有查询接口天然幂等 (GET 请求)
- WES 需对查询结果进行本地缓存，避免短时间内重复调用相同参数的接口

3. WMS → WES 标准回调接口（遵循白皮书规范）

重要: WMS 作为"外部系统"调用 WES，必须使用白皮书定义的标准回调接口

3.1 统一入口：POST /api/v1/callback/event

设计原则（遵循 workline_plugin_architecture_design.md § 6.1）：

- 所有进入编排器的输入统一抽象为 `WorklineInbound`
- Callback API 只做接收、校验、原始落库、ACK、写 Inbox
- 不直接承载复杂业务逻辑

3.2 扩展事件类型（需新增）

在现有 `EventType` 基础上，新增 WMS 相关事件类型：

代码块

```
1  class EventType(str, Enum):
2      """事件类型枚举（白皮书 3.2.2 + WMS 扩展）"""
3
4      # === 设备状态事件（原有） ===
5      ESTOP_PRESSED = "ESTOP_PRESSED"
6      DEVICE_ONLINE = "DEVICE_ONLINE"
7      DEVICE_OFFLINE = "DEVICE_OFFLINE"
8      DEVICE_ERROR = "DEVICE_ERROR"
9
10     # === 业务触发事件（原有） ===
11     MATERIAL_ARRIVED = "MATERIAL_ARRIVED"
12     SCAN_COMPLETED = "SCAN_COMPLETED"
13     PICK_COMPLETED = "PICK_COMPLETED"
14     PUT_COMPLETED = "PUT_COMPLETED"
15     PROCESS_COMPLETED = "PROCESS_COMPLETED"
16
17     # === WMS 系统事件（新增） ===
18     WMS_GRN_RECEIVED = "WMS_GRN_RECEIVED" # GRN 单据接收
19     WMS_PALLET_ARRIVED = "WMS_PALLET_ARRIVED" # 栈板到达通知
20     WMS_RACK_ARRIVED = "WMS_RACK_ARRIVED" # 货架到达通知
21     WMS_INVENTORY_UPDATED = "WMS_INVENTORY_UPDATED" # 库存更新通知
22     WMS_TRANSPORT_COMPLETED = "WMS_TRANSPORT_COMPLETED" # 搬运任务完成
23     WMS_EXCHANGE_COMPLETED = "WMS_EXCHANGE_COMPLETED" # 交换任务完成
```

3.3 事件上报格式（遵循白皮书 3.2.2）

3.3.1 GRN 单据接收

请求：

代码块

```
1  {
```

```
2  "device_code": "WMS_SYSTEM",
3  "event_type": "WMS_GRN_RECEIVED",
4  "timestamp": 1702627300000,
5  "data": {
6    "request_id": "REQ-20260314-001",
7    "grn_id": "GRN.0001",
8    "po_number": "PO-2025-001",
9    "dock_location": "DOCK-01",
10   "allow_mixed_pallet": true,
11   "items": [
12     {
13       "material_id": "CAP001",
14       "material_name": "电容 0402",
15       "qty": 50000,
16       "vendor": "V0001",
17       "lc": "LC01",
18       "dc": "2026-03-14"
19     }
20   ]
21 }
22 }
```

响应:

代码块

```
1  {
2    "code": 200,
3    "message": "Event received",
4    "data": {
5      "status": "submitted",
6      "device_code": "WMS_SYSTEM"
7    }
8  }
```

3.3.2 栈板到达通知

请求:

代码块

```
1  {
2    "device_code": "WMS_RCS",
3    "event_type": "WMS_PALLET_ARRIVED",
4    "timestamp": 1702627300000,
5    "data": {
```

```

6      "pallet_id": "PAL.0001",
7      [{
8          "grn_id": "GRN.0001",
9          "arrival_location": "KITTING_AREA",
10         "arrival_time": "2026-03-14T10:30:00Z",
11         "material_count": 5,
12         "total_qty": 25000,
13         "rcs_task_id": "RCS-TASK-001"
14     },
15     {
16         "grn_id": "GRN.0002",
17         "arrival_location": "KITTING_AREA",
18         "arrival_time": "2026-03-14T10:30:00Z",
19         "material_count": 5,
20         "total_qty": 25000,
21         "rcs_task_id": "RCS-TASK-001"
22     }]
23     }
24 }

```

3.3.3 货架到达通知

请求:

代码块

```

1  {
2      "device_code": "WMS_RCS",
3      "event_type": "WMS_RACK_ARRIVED",
4      "timestamp": 1702627300000,
5      "data": {
6          "rack_id": "RACK-001",           // 货架号
7          "rack_type": "SINGLE_LAYER",     // 货架类型
8          "arrival_location": "KITTING_AREA",
9          "arrival_time": "2026-03-14T10:35:00Z",
10         "rcs_task_id": "RCS-TASK-002"
11     }
12 }

```

3.3.4 搬运/交换任务完成通知

请求:

代码块

```

1  {
2      "device_code": "WMS_RCS",

```

```
3   "event_type": "WMS_TRANSPORT_COMPLETED",
4   "timestamp": 1702627300000,
5   "data": {
6     "rcs_task_id": "RCS-TASK-003",
7     "task_type": "TRANSPORT",
8     "status": "SUCCESS",
9     "rack_id": "RACK-001",
10    "from_location": "KITTING_AREA",
11    "to_location": "SMT_BUFFER"
12  }
13 }
```

3.4 幂等性要求

根据白皮书 4.1 节，WMS 必须为每个事件生成唯一标识：

- `request_id` : 全局请求 ID
- WES 根据 `(device_code + event_type + timestamp + request_id)` 进行幂等检查
- 重复事件返回 200 OK，不重复处理

4. WES → WMS 接口

注意: 这是 WES 主动调用 WMS 的接口，不属于白皮书回调接口范畴

4.1 空架补给请求

接口: `POST /api/wes/rack-supply-request`

请求:

代码块

```
1  {
2    "request_id": "REQ-RACK-001",
3    "area": "KITTING_AREA",
4    "rack_type": "SINGLE_LAYER",
5    "urgency": "HIGH",
6    "reason": "EMPTY_RACK_THRESHOLD"
7  }
```

响应:

代码块

```
1  {
```

```
2     "code": 200,
3     "data": {
4         "task_id": "RCS-TASK-002",
5         "estimated_arrival": "2026-03-14T10:50:00Z"
6     }
7 }
```

4.2 满架搬运请求

接口: `POST /api/wes/transport-request`

请求:

代码块

```
1  {
2     "request_id": "REQ-TRANS-001",
3     "rack_id": "RACK-001",
4     "rack_type": "SINGLE_LAYER",
5     "from_location": "KITTING_AREA",
6     "to_location": "SMT_BUFFER",
7     "priority": 8
8 }
```

4.3 PKG 绑定通知

接口: `POST /api/wms/kitting/pkg-binding`

请求:

代码块

```
1  {
2     "pkg_code": "CAP001-5000-PKG12345-LC01-DC02-V0001",
3     "bin_id": "BIN-001",
4     "slot_id": "SLOT-01",
5     "rack_id": "RACK-001",
6     "grn_id": "GRN.0001",
7     "material_id": "CAP001",
8     "qty": 5000,
9     "vendor": "V0001",
10    "lc": "LC01",
11    "dc": "DC02",
12    "thickness": 12.5
13 }
```

4.4 库存相关接口

接口	方法	用途
GET /api/wms/inventory/query	GET	库存查询
POST /api/wms/inventory/reserve	POST	库存预留
DELETE /api/wms/inventory/reserve/{id}	DELETE	释放预留
POST /api/wms/inventory/transfer	POST	库存转移确认

5. 架构演进路径（遵循 workline_plugin_architecture_design.md）

5.1 统一输入模型

根据设计方案，所有进入编排器的输入统一抽象为 `WorklineInbound`：

代码块

```
1  class InboundKind(str, Enum):
2      DEVICE_EVENT = "DEVICE_EVENT"           # 设备事件
3      COMMAND_RESULT = "COMMAND_RESULT"        # 指令结果
4      EXTERNAL_CALLBACK = "EXTERNAL_CALLBACK"  # 外部系统回调 (WMS)
5      TIMEOUT = "TIMEOUT"                     # 超时
6      MANUAL_OPERATION = "MANUAL_OPERATION"    # 人工操作
7
8
9  class WorklineInbound(BaseModel):
10     inbox_id: int
11     kind: InboundKind
12     source_system: str                    # "WMS" / "ECS" / "RCS" / ...
13     source_message_id: str | None        # WMS request_id
14     workline_id: int | None
15     device_id: int | None
16     command_id: int | None
17     session_id: int | None
18     correlation_id: str | None
19     event_time: datetime
20     payload: dict
```

5.2 接入层职责

Callback API (`callback/event` , `callback/result`) 只做:

1. **请求校验**: 验证请求格式、权限
2. **原始日志落库**: 记录 `DeviceEventLog` / `DeviceCommand`
3. **写 WorklineInbox**: 将输入持久化到 Inbox
4. **立即 ACK**: 快速返回响应

不做:

- 执行业务决策
- 直接改 Session
- 直接发设备命令

5.3 编排层职责

`WorklineOrchestrator` 负责:

1. 消费 `WorklineInbox`
2. 解析 `device -> workline -> plugin -> mode`
3. 创建/恢复 `WorklineSession`
4. 调用插件进行业务决策
5. 原子写入 `Session / Timeline / Decision / Outbox`

6. 装箱区接口调用时序

代码块

```
1  sequenceDiagram
2      participant WMS
3      participant WES_Callback as WES /callback/event
4      participant WES_Inbox as WES WorklineInbox
5      participant WES_Orchestrator as WES Orchestrator
6      participant ECS
7      participant RCS
8
9      Note over WMS,WES_Callback: 1. GRN 单据接入 (通过标准回调接口)
10     WMS->>WES_Callback: POST /api/v1/callback/event {event_type:
        "WMS_GRN_RECEIVED", ...}
11     WES_Callback->>WES_Inbox: 写入 Inbox (kind=EXTERNAL_CALLBACK)
12     WES_Callback-->>WMS: 200 OK (立即返回)
13     WES_Orchestrator->>WES_Inbox: 消费 Inbox
14     WES_Orchestrator->>WES_Orchestrator: 解析 -> 创建 Session
15
```

```
16     Note over WMS,WES_Callback: 2. 栈板到达通知
17     WMS->>WES_Callback: POST /api/v1/callback/event
18     {event_type: "WMS_PALLET_ARRIVED", ...}
19     WES_Callback-->>WMS: 200 OK
20
21     Note over WES_Orchestrator,WMS: 3. 空架补给请求
22     WES_Orchestrator->>WMS: POST /api/wes/rack-supply-request
23     WMS->>RCS: 调度 AGV
24     RCS-->>WMS: 完成
25
26     Note over WMS,WES_Callback: 4. 货架到达通知
27     WMS->>WES_Callback: POST /api/v1/callback/event
28     {event_type: "WMS_RACK_ARRIVED", ...}
29     WES_Callback-->>WMS: 200 OK
30
31     Note over ECS,WES_Callback: 5. 设备事件（视觉扫描）
32     ECS->>WES_Callback: POST /api/v1/callback/event
33     {event_type: "SCAN_COMPLETED", ...}
34     WES_Callback-->>ECS: 200 OK
35
36     Note over WES_Orchestrator,WMS: 6. PKG 绑定通知
37     WES_Orchestrator->>WMS: POST /api/wms/kitting/pkg-binding
38
39     Note over WES_Orchestrator,WMS: 7. 满架搬运请求
40     WES_Orchestrator->>WMS: POST /api/wes/transport-request
41     WMS->>RCS: 调度 AGV
42     RCS-->>WMS: 完成
43
44     Note over WMS,WES_Callback: 8. 搬运完成通知
45     WMS->>WES_Callback: POST /api/v1/callback/event
46     {event_type: "WMS_TRANSPORT_COMPLETED", ...}
```

7. 接口清单汇总

7.1 WMS 调用 WES 的接口（通过标准回调入口）

接口	事件类型	用途	优先级
POST /api/v1/callback/event	WMS_GRN_RECEIVED	GRN 单据接收	P0
POST /api/v1/callback/event	WMS_PALLET_ARRIVED	栈板到达通知	P0

POST /api/v1/callback/event	WMS_RACK_ARRIVED	货架到达通知	P0
POST /api/v1/callback/event	WMS_TRANSPORT_COMPLETED	搬运任务完成	P0
POST /api/v1/callback/event	WMS_EXCHANGE_COMPLETED	交换任务完成	P1
POST /api/v1/callback/result	-	RCS 任务结果回传	P0

7.2 WES 调用 WMS 的接口

7.2.1 基础数据查询接口

接口	方法	用途	优先级	调用场景
/api/wms/materials/{material_id}	GET	查询单个物料主数据	P0	PKG 校验、分箱算法
/api/wms/materials?ids={id1,id2}	GET	批量查询物料主数据	P1	分箱预计算、波次查询
/api/wms/zones	GET	查询区域列表	P1	设备归属解析、初始化
/api/wms/locations?zone={zone}	GET	查询区域内地码	P1	货架位置管理
/api/wms/racks/{rack_id}	GET	查询单个货架信息	P0	货架状态初始化
/api/wms/bins/{bin_id}	GET	查询单个料箱信息	P0	料箱属性查询
/api/wms/racks?type={type}	GET	按类型查询货架	P0	混合入库查找空箱
/api/wms/grn/{grn_id}	GET	查询单个 GRN	P0	PKG 校验时验证 GRN

<code>/api/wms/grn/{grn_id}/packages</code>	GET	查询 GRN 下的料盘列表	P0	PKG 校验时匹配物料
<code>/api/wms/inventory/query</code>	GET	库存查询	P0	发料决策、库存校验

7.2.2 业务指令接口

接口	方法	用途	优先级
<code>/api/wes/rack-supply-request</code>	POST	空架补给请求	P0
<code>/api/wes/transport-request</code>	POST	满架搬运请求	P0
<code>/api/wms/kitting/pkg-binding</code>	POST	PKG 绑定通知	P0
<code>/api/wms/inventory/reserve</code>	POST	库存预留	P1
<code>/api/wms/inventory/reserve/{id}</code>	DELETE	释放预留	P1
<code>/api/wms/inventory/transfer</code>	POST	库存转移确认	P1

7.3 需要新增的 EventType

代码块

```
1  # 在 src/app/device/models/event_log.py 中新增
2  WMS_GRN_RECEIVED = "WMS_GRN_RECEIVED"
3  WMS_PALLET_ARRIVED = "WMS_PALLET_ARRIVED"
4  WMS_RACK_ARRIVED = "WMS_RACK_ARRIVED"
5  WMS_INVENTORY_UPDATED = "WMS_INVENTORY_UPDATED"
6  WMS_TRANSPORT_COMPLETED = "WMS_TRANSPORT_COMPLETED"
7  WMS_EXCHANGE_COMPLETED = "WMS_EXCHANGE_COMPLETED"
```

8. 实施建议

8.1 第一阶段：扩展 EventType

- 1. 在 `src/app/device/models/event_log.py` 中新增 WMS 相关事件类型
- 2. 更新 `device_event_log` 表的数据验证

8.2 第二阶段：实现 WorklineInbox

根据 `workline_plugin_architecture_design.md`：

- 1. 新增 `WorklineInbox` 表
- 2. 修改 `callback/event` 接口，写入 Inbox
- 3. 实现 `WorklineOrchestrator` 消费 Inbox

8.3 第三阶段：WMS 适配

- 1. WMS 侧定义虚拟设备：`WMS_SYSTEM`，`WMS_RCS`
- 2. WMS 调用 WES 时使用标准回调接口
- 3. WMS 生成 `request_id` 保证幂等性

9. 附录：错误码定义

错误码	说明
<code>WMS_GRN_NOT_FOUND</code>	GRN 不存在
<code>WMS_MATERIAL_NOT_FOUND</code>	物料不存在
<code>WMS_PKG_ALREADY_BOUND</code>	PKG 已绑定
<code>WMS_PKG_NOT_IN_GRN</code>	PKG 不属于该 GRN
<code>WMS_INSUFFICIENT_INVENTORY</code>	库存不足
<code>WMS_RCS_DISPATCH_FAILED</code>	RCS 调度失败
<code>WMS_API_TIMEOUT</code>	WMS API 超时
<code>DUPLICATE_EVENT</code>	重复事件（幂等拦截）

10. 附录：WMS/WES 开发人员参考

10.1 WMS 侧开发要点

事件上报要求:

1. 所有调用 WES `callback/event` 和 `callback/result` 的请求必须携带唯一
2. `request_id` 格式建议: `REQ-{YYYYMMDD}-{SEQ}`, 如 `REQ-20260314-001`
3. 若 WES 返回 200 OK, 表示事件已接收; 若返回 4xx/5xx, WMS 需记录失败并重试
4. 重复事件会被 WES 拦截, WMS 不应依赖重复上报来保证可靠性

数据一致性:

- WMS 是库存唯一真实源, WES 所有库存查询都实时透传 WMS
- WMS 需在库存变动事务提交成功后, 方可通知 WES 执行后续动作
- 若 WMS 与 WES 断网, WES 将暂停所有涉及库存变动的任务

10.2 WES 侧开发要点

查询接口调用:

1. 所有基础数据查询接口支持短时缓存 ($TTL \leq 30$ 秒)
2. 缓存失效后必须重新查询 WMS, 不得使用过期数据
3. 若 WMS 接口超时或返回 5xx, WES 需触发熔断并告警

编排器集成:

1. 所有 WMS 回调事件必须写入 `WorklineInbox`, 不得直接处理
2. 编排器根据 `workline_id + business_key` 创建/恢复 `WorklineSession`
3. 业务决策必须由插件输出 `PluginResult`, 编排器统一生成 Timeline/Outbox

10.3 联调测试建议

基础数据接口测试:

- ☐ 物料查询: 存在/不存在/批量查询
- ☐ 区域/地码查询: 有效区域/无效区域
- ☐ 货架/料箱查询: 空架/满架/部分占用
- ☐ GRN 查询: 存在/不存在/已关闭 GRN
- ☐ 库存查询: 有库存/无库存/部分预占

回调接口测试:

- ☐ GRN 单据接收: 正常接收/重复接收 (幂等)
- ☐ 栈板/货架到达: 正常接收/参数缺失
- ☐ 搬运完成: 成功/失败/超时

☐ 事件幂等：相同 request_id 重复调用

异常场景测试:

☐ WMS 接口超时：WES 熔断和告警

☐ 网络中断：WES 暂停库存相关任务

☐ 数据不一致：WES 校验失败并上报